# GPO AIG

MetasploitMCP on Kali Linux - Printable Reference Manual

Use only on systems you own or are explicitly authorized to test. This guide starts with discovery, checks, safe enumeration, job monitoring, and session visibility.

## Purpose and operating boundary

This manual configures a Kali Linux Dell Latitude as the **security-testing host **and your Mac as the** control/AI client**. Use it only against systems you own or have explicit permission to test.

The initial profile enables the safer first half of the bridge:

- discover Metasploit modules and payload names;
- inspect module options;
- check a target before exploitation;
- run authorized auxiliary enumeration;
- monitor background jobs;
- view existing sessions.

Do not begin with exploitation, payload generation, listeners, or post-exploitation. Those capabilities remain installed but are intentionally not part of the starter workflow.

## 1. Machine roles

| Machine | Role | What runs there |
|---|---|---|
| Kali Dell Latitude | Execution host | Metasploit Framework, `msfrpcd`, MetasploitMCP |
| Mac | Control/client host | Copilot or another MCP client; browser/terminal for administration |
| Lab target | Authorized target | Intentionally vulnerable VM, test server, or isolated device |
| Optional lab network | Isolation boundary | Host-only/NAT network; no public exposure |

Use the **Kali Dell** for every Metasploit operation. Use the **Mac** only to send approved MCP requests and administer Kali over SSH. Do not run MetasploitMCP on both computers at the same time.

## 2. Recommended modern topology

### Same computer as the MCP client

Use this when the MCP client runs directly on Kali:

```
MCP client -> 127.0.0.1:8085 -> MetasploitMCP -> 127.0.0.1:55553 -> msfrpcd
                                      |
                                      +-> authorized lab target
```

This is the safest and simplest arrangement. Bind both services to loopback.

### Mac controlling Kali over a private network

Use this when Copilot runs on the Mac and Metasploit runs on Kali:

```
Mac MCP client
      |
      | SSH or Tailscale/WireGuard private link
      v
Kali:8085 -> MetasploitMCP -> Kali:55553 -> authorized lab target
```

Prefer **Tailscale or WireGuard** over port forwarding. Never expose port 55553 or 8085 to the public internet. If HTTP/SSE is bound beyond loopback, use `MCP_AUTH_TOKEN` and restrict the firewall to the private interface.

## 3. Identify the correct computer and IP addresses

Run these commands in a terminal **on the Kali Dell**:

```
hostname
uname -a
ip -br address
```

Record the Kali private address, for example `192.168.56.20` or a Tailscale address such as `100.x.y.z`.

Run this on the **Mac** only to test reachability:

```
ping -c 3 KALI_IP
ssh kali-user@KALI_IP
```

Replace `KALI_IP` and `kali-user` with the actual values. If SSH is not installed or enabled on Kali, enable it locally on Kali:

```
sudo apt update
sudo apt install -y openssh-server
sudo systemctl enable --now ssh
```

For a modern key-based login, create a key on the Mac and copy it to Kali:

```
ssh-keygen -t ed25519 -C "mac-to-kali"
ssh-copy-id kali-user@KALI_IP
ssh kali-user@KALI_IP
```

Use a private network or VPN address instead of a public address.

## 4. Install the stack on Kali

Run the following in **Terminal 1 on Kali**:

```
sudo apt update
sudo apt install -y git python3 python3-venv python3-pip \
  openssl curl iproute2 net-tools metasploit-framework

mkdir -p "$HOME/tools"
cd "$HOME/tools"

if [ ! -d MetasploitMCP ]; then
  git clone https://github.com/XOF34/MetasploitMCP.git
fi

cd MetasploitMCP
python3 -m venv .venv
. .venv/bin/activate
```

```
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
mkdir -p "$HOME/payloads"
```

If Kali's package repository does not provide `metasploit-framework`, install

Metasploit using Rapid7's official Kali-supported installation instructions,

then repeat the verification in the next section.

## 5. Configure secrets and local paths

Run this in **Terminal 1 on Kali**:

```
cd "$HOME/tools/MetasploitMCP"

cat > "$HOME/.metasploitmcp.env" <<'EOF'
export MSF_SERVER=127.0.0.1
export MSF_PORT=55553
export MSF_SSL=false
export MSF_ENABLED=true
export PAYLOAD_SAVE_DIR="$HOME/payloads"
export MCP_AUTH_TOKEN="REPLACE_WITH_A_LONG_RANDOM_TOKEN"
EOF

TOKEN="$(openssl rand -hex 32)"
sed -i "s/REPLACE_WITH_A_LONG_RANDOM_TOKEN/$TOKEN/" "$HOME/.metasploitmcp.env"
chmod 600 "$HOME/.metasploitmcp.env"

printf 'Generated MCP token: %s\n' "$TOKEN"
printf 'Store it in your password manager; do not commit it to Git.\n'
```

The RPC password is generated separately when `msfrpcd` starts. Do not paste

either secret into a public chat, issue, or repository.

## 6. Start Metasploit RPC on Kali

Open **Terminal 2 on Kali**:

```
set -a
. "$HOME/.metasploitmcp.env"
set +a

export MSF_PASSWORD="$(openssl rand -hex 24)"
printf 'MSF_PASSWORD=%s\n' "$MSF_PASSWORD"
printf 'Keep this terminal open while the service is running.\n'

msfrpcd -P "$MSF_PASSWORD" \
  -a 127.0.0.1 \
  -p "$MSF_PORT" \
  >"$HOME/msfrpcd.log" 2>&1
```

Leave Terminal 2 open. If the command exits, inspect the log:

```
tail -n 100 "$HOME/msfrpcd.log"
```

In **Terminal 3 on Kali**, verify the RPC listener:

```
ss -lntp | grep ':55553'
```

Expected result: a listener on `127.0.0.1:55553`.

## 7. Start MetasploitMCP on Kali

Run this in **Terminal 4 on Kali**:

```
cd "$HOME/tools/MetasploitMCP"
. .venv/bin/activate
set -a
. "$HOME/.metasploitmcp.env"
```

```
set +a
export MSF_PASSWORD='PASTE_THE_PASSWORD_PRINTED_IN_TERMINAL_2'

python MetasploitMCP.py \
  --transport http \
  --host 127.0.0.1 \
  --port 8085
```

For a client running on the Mac over a private VPN, use Kali's private address instead:

```
python MetasploitMCP.py \
  --transport http \
  --host KALI_PRIVATE_IP \
  --port 8085
```

Only use the second form when `KALI_PRIVATE_IP` is reachable through a private network or VPN and the MCP token is configured.

# 8. Verify the service before using an MCP client

Run on **Kali**:

```
ss -lntp | grep ':8085'
curl -i http://127.0.0.1:8085/
```

If the MCP client is on the Mac and the server is bound to the private Kali address, test from the **Mac**:

```
curl -i http://KALI_PRIVATE_IP:8085/
```

A connection failure means the service is not listening, the host is not reachable, or the firewall is blocking it. Do not immediately expose the port publicly.

# 9. Configure the MCP client

## Client on Kali

Use this endpoint:

```
http://127.0.0.1:8085/sse
```

## Client on the Mac

Use this endpoint:

```
http://KALI_PRIVATE_IP:8085/sse
```

Send the token as either:

```
Authorization: Bearer MCP_AUTH_TOKEN
```

or:

```
X-MCP-Token: MCP_AUTH_TOKEN
```

Use the exact token from `$HOME/.metasploitmcp.env` on Kali. Do not place the token in a Git repository or a shared screenshot.

# 10. First-use workflow: safe 50% profile

Use these prompts from the **Mac MCP client** or the MCP client on Kali.

Replace the target with an authorized lab address.

### Step 1: Confirm scope

```
I am testing 192.168.56.10, an intentionally vulnerable lab VM that I own.
Only perform discovery and vulnerability checks. Do not exploit, generate
payloads, start listeners, execute post modules, or alter the target.
```

### Step 2: Discover modules

```
List Metasploit exploits related to SMB. Return module names only.
Do not run a module.
```

```
List auxiliary modules relevant to HTTP service discovery.
Do not run a module.
```

### Step 3: Inspect options

```
For the selected authorized module, show the required options and explain
which values are safe for my lab. Do not execute it.
```

### Step 4: Check before acting

```
Run only the module's vulnerability check against 192.168.56.10.
Do not exploit the host. Report the check result and any missing options.
```

### Step 5: Run safe enumeration as a job

```
Run an authorized, non-exploit auxiliary service-discovery module against
192.168.56.10 as a background job. Return the job ID and do not start a
handler or create a payload.
```

### Step 6: Monitor

```
List active Metasploit jobs and sessions. Do not terminate or modify anything.
```

### Step 7: Stop the job

```
Stop job JOB_ID after confirming it is the enumeration job I started.
```

## 11. What is included and excluded

### Included in the starter profile

- exploit and payload discovery;
- module option review;
- vulnerability checks;
- authorized auxiliary scanning and enumeration;
- background job monitoring;
- listing existing sessions;
- service and connectivity troubleshooting.

### Excluded until separately approved

- `run_exploit`;
- `generate_payload`;
- `start_listener`;
- `run_post_module`;
- `send_session_command`;
- `terminate_session`.

These are not removed from the server. They require a separate, explicit

decision because they can change systems, create network connections, or execute commands.

# 12. Modern operating practices

## Prefer private networking

Use Tailscale or WireGuard between the Mac and Kali instead of router port forwarding. Bind Metasploit RPC to `127.0.0.1` even when MCP HTTP is reachable over the private link.

## Prefer SSH keys

Use Ed25519 SSH keys and disable password SSH after confirming key login:

```
sudoedit /etc/ssh/sshd_config
```

Set:

```
PasswordAuthentication no
PermitRootLogin no
```

Then apply:

```
sudo systemctl reload ssh
```

Do this only after verifying a second key-based SSH session works.

## Use snapshots and isolated targets

Use an intentionally vulnerable VM on a host-only or isolated NAT network.

Take a VM snapshot before testing and revert it after the exercise.

## Keep evidence separate

Store scan output and notes in a dated directory outside the Git repository:

```
mkdir -p "$HOME/lab-evidence/$(date +%Y-%m-%d)"
chmod 700 "$HOME/lab-evidence"
```

## Do not run as root by default

Run MetasploitMCP and `msfrpcd` as your normal Kali user. Use `sudo` only for package installation, SSH configuration, firewall configuration, or other administrative tasks.

# 13. Troubleshooting

## RPC port is not listening

On Kali:

```
ss -lntp | grep ':55553' || true
tail -n 100 "$HOME/msfrpcd.log"
```

Check that the password used by `MetasploitMCP.py` exactly matches the password passed to `msfrpcd`.

## MCP port is not listening

On Kali:

```
ss -lntp | grep ':8085' || true
```

Confirm the virtual environment is active and that `requirements.txt`

installed successfully.

### Mac cannot reach Kali

From the Mac:

```
ping -c 3 KALI_PRIVATE_IP
ssh kali-user@KALI_PRIVATE_IP
curl -i http://KALI_PRIVATE_IP:8085/
```

If SSH works but HTTP does not, check the bind address and Kali firewall.

### Firewall configuration

If you intentionally use a private interface, allow only the private source

address. Example on Kali with UFW:

```
sudo ufw allow from MAC_PRIVATE_IP to any port 8085 proto tcp
sudo ufw deny 55553/tcp
sudo ufw enable
sudo ufw status verbose
```

Do not copy this rule unchanged if your network addresses differ.

### Architecture or framework errors

On Kali:

```
command -v msfconsole
msfconsole --version
python --version
```

If Metasploit is missing, repair the official framework installation before

debugging the MCP bridge.

## 14. Shutdown and cleanup

Stop MetasploitMCP with `Ctrl-C` in Terminal 4. Stop `msfrpcd` with `Ctrl-C`

in Terminal 2. Then verify that no service remains:

```
ss -lntp | grep -E ':(55553|8085)' || true
```

The cleanup prompt for the MCP client is:

```
Show active jobs and sessions, then stop only the jobs created during this
lab exercise. Do not terminate unrelated sessions.
```

## 15. Quick reference

| Item | Default |
|---|---|
| Kali RPC bind | `127.0.0.1:55553` |
| MCP HTTP bind | `127.0.0.1:8085` |
| MCP endpoint | `http://HOST:8085/sse` |
| Framework | Metasploit Framework |
| MCP server | `MetasploitMCP.py` |
| Python environment | `$HOME/tools/MetasploitMCP/.venv` |
| Payload directory | `$HOME/payloads` |
| Evidence directory | `$HOME/lab-evidence/YYYY-MM-DD` |
| Secrets file | `$HOME/.metasploitmcp.env` |